VULNERABILITY ASSESSMENT REPORT

Fictional training example • Food delivery and e-commerce application scenarios • 18 October 2026

TRAINING SAMPLE — NOT A REAL SCAN
The applications, findings, evidence and risk ratings in this document are fictional examples created for learning. No real Zomato- or Flipkart-owned systems were scanned.

1. Executive Summary

A hypothetical security review was considered for a food delivery application that stores customer profiles, delivery addresses, order history and payment-related records. For this training scenario, the overall risk is **High** because weaknesses could expose private customer information and undermine trust in the service. The example findings include a possible database query weakness and inadequate access checks on order records. If exploited, these issues could lead to data exposure, privacy complaints, financial or operational costs, and reputational damage. The development team should prioritize fixing high-impact input-handling and authorization issues, then retest the fixes before release. These are illustrative risks, not verified findings from an actual scan.

2. Findings and Recommendations

VA-01 — SQL Injection (fictional e-commerce search endpoint)

Risk rating	**High (illustrative)**
Description / impact	If user-supplied search or filter text is joined directly into a database query, specially crafted input may alter the query. Depending on database permissions and implementation, this could expose or modify customer, product, or order data.

Sample output	Illustrative only — not captured from a real scan:
	`Test case: search input handling`
	`Expected: input treated as data`
	`Example observation: unsafe query construction suspected`
	`Status: requires validation in an authorized test environment`

Recommendation	Use parameterized queries or prepared statements for every database operation. Avoid building SQL by concatenating user input, validate input with allow-lists where appropriate, apply least-privilege database accounts, and add regression tests for search and filter fields.

VA-02 — Insecure Direct Object Reference (IDOR) / Broken Object-Level Authorization

Risk rating	**High (illustrative)**
Description / impact	A customer-facing endpoint may return an order when its identifier is changed, without verifying that the signed-in user owns that order. This could expose another customer's delivery details or purchase history.

Sample output	Illustrative only — fictional response summary:
	`Request: GET /api/orders/ORDER_ID`
	`Expected: only the owner's order is returned`
	`Example observation: ownership check not demonstrated`
	`Status: requires validation in an authorized test environment`

Recommendation	Enforce server-side authorization on every request. Check that the authenticated user is permitted to access the specific order before returning data; do not rely on hidden buttons or unpredictable IDs. Add automated tests using two separate test accounts and log denied access attempts without recording sensitive data.

3. Risk Summary

ID	Finding	Rating	Priority
VA-01	SQL Injection	High*	Fix promptly; verify with safe tests
VA-02	IDOR / authorization failure	High*	Fix promptly; test ownership checks

*Ratings are illustrative and should be assigned using the actual likelihood, impact, scope and evidence from an authorized assessment.

4. Appendix — Scope, Method and Reference

• **Scope:** Fictional food delivery app for the executive summary and fictional e-commerce API for the two example findings.
• **Method:** No live testing was performed for this assignment. Sample outputs above are mock evidence and must not be presented as real scanner output or screenshots.
• **Risk scale:** High = potential serious exposure or unauthorized access; Medium = meaningful but more limited impact; Low = limited impact. Final ratings depend on context and validation.
• **OWASP reporting guidance:** OWASP Web Security Testing Guide — Reporting. OWASP recommends an executive summary, clear findings, remediation details and relevant supporting evidence.